Practical Week 12

```
samanth-ubuntu@samanth-ubuntu:~$ nano producer_consumer.c
samanth-ubuntu@samanth-ubuntu:~$ gcc producer_consumer.c -o producer_consumer -pthread
samanth-ubuntu@samanth-ubuntu:~$ ./producer_consumer 2
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 2
Time taken  : 3.280342 seconds
Throughput  : 30484.63 items/second
samanth-ubuntu@samanth-ubuntu:~$ ./producer_consumer 10
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 10
Time taken  : 0.682802 seconds
Throughput  : 146455.35 items/second
samanth-ubuntu@samanth-ubuntu:~$ ./producer_consumer 50
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 50
Time taken  : 0.029425 seconds
Throughput  : 3398489.75 items/second
samanth-ubuntu@samanth-ubuntu:~$ ./producer_consumer 100
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 100
Time taken  : 0.026453 seconds
Throughput  : 3780286.14 items/second
samanth-ubuntu@samanth-ubuntu:~$ ./producer_consumer 1000
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 1000
Time taken  : 0.012609 seconds
Throughput  : 7930929.85 items/second
samanth-ubuntu@samanth-ubuntu:~$ nano deadlock.c
samanth-ubuntu@samanth-ubuntu:~$ gcc deadlock.c -o deadlock -pthread
samanth-ubuntu@samanth-ubuntu:~$ ./deadlock
```

```
Thread 1: locking Resource A
Thread 1: Resource A acquired
Thread 2: locking Resource B
Thread 2: Resource B acquired
Thread 1: waiting for Resource B
Thread 2: waiting for Resource A

DEADLOCK DETECTED!
Both threads are waiting for resources held by each other.
Program terminated.
samanth-ubuntu@samanth-ubuntu:~$ nano deadlock_prevention.c
samanth-ubuntu@samanth-ubuntu:~$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
samanth-ubuntu@samanth-ubuntu:~$ ./deadlock_prevention
Thread 1: locking Resource A
Thread 1: Resource A acquired
Thread 2: locking Resource A
Thread 1: locking Resource B
Thread 1: Resource B acquired
Thread 1: using both resources
Thread 1: resources released
Thread 2: Resource A acquired
Thread 2: locking Resource B
Thread 2: Resource B acquired
Thread 2: using both resources
Thread 2: resources released

No deadlock occurred.
Both threads followed the resource ordering A -> B.
samanth-ubuntu@samanth-ubuntu:~$ gcc producer_consumer.c -o producer_consumer -pthread

./producer_consumer 2
./producer_consumer 10
./producer_consumer 50
./producer_consumer 100
./producer_consumer 1000

gcc deadlock.c -o deadlock -pthread
./deadlock

gcc deadlock_prevention.c -o deadlock_prevention -pthread
```

```
No deadlock occurred.
Both threads followed the resource ordering A -> B.
samanth-ubuntu@samanth-ubuntu:~$ gcc producer_consumer.c -o producer_consumer -pthread
./producer_consumer 2
./producer_consumer 10
./producer_consumer 50
./producer_consumer 100
./producer_consumer 1000

gcc deadlock.c -o deadlock -pthread
./deadlock

gcc deadlock_prevention.c -o deadlock_prevention -pthread
./deadlock_prevention
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 2
Time taken  : 3.315989 seconds
Throughput  : 30156.91 items/second
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 10
Time taken  : 0.637791 seconds
Throughput  : 156791.24 items/second
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 50
Time taken  : 0.024164 seconds
Throughput  : 4138319.87 items/second
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 100
Time taken  : 0.020953 seconds
Throughput  : 4772500.62 items/second
Consumer processed 100000 items.
Consumer sum = 4999950000

Buffer size : 1000
Time taken  : 0.012522 seconds
Throughput  : 7986096.53 items/second
Thread 1: locking Resource A
Thread 1: Resource A acquired
Thread 2: locking Resource B
Thread 2: Resource B acquired
Thread 1: waiting for Resource B
Thread 2: waiting for Resource A

DEADLOCK DETECTED!
Both threads are waiting for resources held by each other.
Program terminated.
Thread 1: locking Resource A
Thread 1: Resource A acquired
Thread 2: locking Resource A
Thread 2: locking Resource B
Thread 1: Resource B acquired
Thread 1: using both resources
Thread 1: resources released
Thread 2: Resource A acquired
Thread 2: locking Resource B
Thread 2: Resource B acquired
Thread 2: using both resources
Thread 2: resources released

No deadlock occurred.
Both threads followed the resource ordering A -> B.
samanth-ubuntu@samanth-ubuntu:~$
```